

Receipt OCR & Information Extraction API

End-to-End AI System for Receipt Extraction

Author

Reiner Jasin

Deployment Target

Self-hosted VPS (Hostinger)

1. Executive Summary

Businesses often store transaction data in handwritten paper receipts. Manual data entry is slow, expensive, and error-prone. This project builds an end-to-end AI system that converts receipt images into structured digital data and exposes the result through REST API, allowing client applications to upload receipt images and receive structured JSON outputs of the receipt which may include:

- Date of transaction
- Customer's name
- Customer's address (for shipping or delivery)
- Customer's phone number
- Purchased items
- Prices of the items
- Total Purchase

The entire system is built using open-source technologies and deployed on a self-managed VPS to minimize operational costs. No paid third-party OCR or AI APIs are used. The solution is suitable for small to medium businesses that require receipt digitization without recurring per-request fees.

The objective of this project is to automatically extract structured data from receipt images which support Indonesian and English. The model is accessible through the API with complete documentation. The model used is also an open-source model without SaaS scheme to prevent unexpected cost.

2. System Overview and Scope

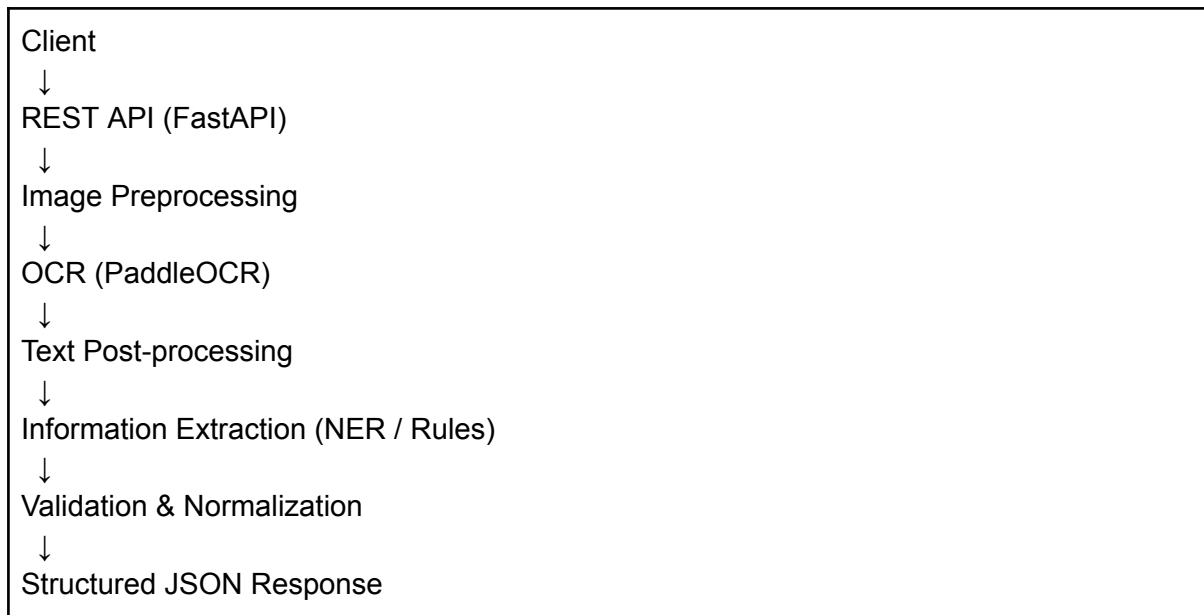
Input:

- Receipt image (JPG / PNG / JPEG / HEIC)
- Uploaded via API

Output:

- Structured JSON response (detail on

Architecture



The API is stateless and processes each request independently. All computation is performed on the VPS without reliance on external services. Clients can then utilize this API on their websites which then send requests in a form of image to be digitized to the API endpoint.

3. Design Goals and Constraints

Design Goals

- Convert receipt images into structured data
- Provide a simple and predictable API interface
- Use only open-source tools
- Enable low-cost deployment on a VPS
- Keep inference CPU-friendly

Constraints

- No paid OCR or AI services
- Limited hardware resources
- Real-world receipt image noise (blur, skew, low contrast)

4. Technology Stack

Component	Technology	Reason
API Framework	FastAPI	• High performance • Auto generated docs
Image Processing	OpenCV (CV2) Numpy alternative: Pillow (PIL)	• Industry-standard image operations

OCR Engine	PaddleOCR alternative: Easy OCR Tesseract OCR	• Best text detection among open-source OCR • Robust to skew, overlap, uneven spacing • Produces accurate bounding boxes and line-level structure • Fully offline • Strong community support • Easy to version-lock models
NLP / Parsing	spaCy, regex	Lightweight text processing
ML Framework	PyTorch	Flexibility for future model training
Serving	Uvicorn	Simple ASGI server
Containerization	Docker	Reproducible deployment
Version Control	Git	Source code tracking
Hosting	Hostinger VPS	Low-cost self-managed server

5. Dataset Description

The dataset consists of receipt images collected from public sources and manually captured photos.

Characteristics:

- Image formats: JPG, PNG, JPEG, HEIC
- Receipt type: Handwritten invoice with printed layout
- Language: Indonesian, English
- Currency: IDR
- Capture conditions:
 - Phone camera images
 - Variable lighting
 - Skewed and partially blurred images

Ground truth annotations include:

- Raw text transcription
- Structured fields:
 - Customer's name | Required
 - Customer's phone number | Optional
 - Customer's address | Optional
 - Purchased items | Required
 - Individual item prices | Required
 - Totals | Required
 - Tax | Optional
 - Service | Optional

6. Image Preprocessing Pipeline

Before OCR inference, images are automatically preprocessed to improve text recognition accuracy.

Preprocessing steps include:

1. Resize image to a fixed width
2. Convert to grayscale
3. Apply noise reduction
4. Perform adaptive thresholding
5. Correct skew using contour analysis

Tools:

- OpenCV (cv2)
- Numpy

Notes:

These steps are applied automatically inside the API and are transparent to the client.

7. OCR Component

The OCR stage extracts raw text from the preprocessed images.

Model: PaddleOCR

Source: <https://github.com/PaddlePaddle/PaddleOCR>

Configuration:

Detection Model: DBNet

Recognition model: CRNN / PP-OCRv4

Language: en + id

OCR Output:

1. Recognized text
2. Bounding Box
3. Confidence Score

Example expected OCR Output:

```
[
  {
    "text": "TOKO MAKMUR",
    "bbox": [34, 18, 420, 58],
    "confidence": 0.96
  },
  {
    "text": "TOTAL 42.000",
    "bbox": [50, 780, 520, 820],
    "confidence": 0.92
  }
]
```

```
}  
]
```

8. Information Extraction

The raw OCR text is converted into structured data using rule-based logic.

Extraction Plan:

- Group OCR lines by vertical position
- Identify items lines containing both text and numeric values
- Detect totals using keywords such as TOTAL, AMOUNT
- Parse dates and currency symbols

Tools:

- Python regex
- spaCy
- Pandas

9. Output Validation

After extraction, logical validation is performed to ensure data consistency.

Validation rules:

- Sum of item prices should match total exactly
- Missing field flagged
- Confidence score below threshold 0.45 (45%) will be flagged

Note:

Validation rules that are not passed will still return the value during API call, but it will include error code and warning messages.

10. API Design

Endpoint

```
POST /v1/ocr/receipt
```

Request

- **Content-Type:** `multipart/form-data`
- **Field:** `file` (receipt image)

Example Request

```
curl -X POST http://api.anekadieselsinjai.com/v1/ocr/receipt \  
-F "file=@nota.jpg"
```

11. API Response Format

Good Example Response

```
{
  "customer_name": "ABC Store",
  "date": "2024-10-01",
  "items": [
    {"name": "Milk", "qty": 2, "price": 15000},
    {"name": "Bread", "qty": 1, "price": 12000}
  ],
  "total": 42000,
  "currency": "IDR",
  "confidence": 0.89,
}
```

Error Example Response

```
{
  "customer_name": null,
  "date": null,
  "items": [
    {"name": "Milk", "qty": 2, "price": 15000},
    {"name": "Bread", "qty": 1, "price": 12000}
  ],
  "total": 57000,
  "currency": "IDR",
  "confidence": 0.36,

  # ERROR FIELD
  "error_code": ["FALSE_NUMBER", "MISSING_FIELD",
    "LOW_CONFIDENCE"],

  # FALSE_NUMBER case
  "error_message": "items prices don't match the total.
Predicted:
                    57000, calculated: 42000",

  # MISSING_FIELD case
  "error_message": "customer_name, date field is missing!",

  # LOW_CONFIDENCE case
  "error_message": "predicted text is below optimal threshold,
                    please recheck the result before
                    confirmation",
}
```

12. API Documentation

The API provides automatically generated documentation using FastAPI.

Available Endpoints

- /docs : Swagger UI

- `/redoc` : ReDocs interface

The documentation includes:

- Endpoint definitions
- Request parameters
- Response schemas
- Example payloads

13. Deployment Architecture

The system is deployed as a single Dockerized service on a self-hosted VPS. All components (API, image processing, OCR, extraction logic) run within one container to minimize complexity and operational overhead.

Process Model

- The OCR model is loaded once at application startup
- Requests reuse the same OCR instance to avoid repeated initialization cost
- The API is stateless; no session data is stored between requests

Concurrency Handling

- The API uses Uvicorn workers to handle concurrent requests
- Requests are processed synchronously
- Maximum concurrent requests are limited based on VPS CPU capacity

Resource Constraints

- CPU-only inference
- Memory usage capped using Docker resource limits
- Large images are resized to prevent excessive memory usage

Failure Handling

- OCR or parsing failures return structured error responses
- Application crashers trigger container restart
- Invalid inputs are rejected early during request validation

14. Monitoring and Logging

Basic monitoring is implemented using application-level logging.

Logged Fields:

- Request timestamps
- Request ID
- Processing duration
- OCR confidence scores
- Error messages

Logs are stored locally and rotated periodically, old logs should be automatically archived, and no sensitive client data will be stored in logs.

15. Security and Privacy

The system is designed with basic security considerations.

Measures:

- File type validation (only accepts image file type of extension)
- File size limits (reject files that is irrationally large)
- Temporary image deletion after processing
- No long-term storage of client data

16. Evaluation Metrics

The evaluation metrics are picked both for technical performance and business impact.

Technical Metrics:

- OCR Performance
 - CER: Character Error Rate
 - WER: Word Error Rate
- Extraction Accuracy
 - Field Accuracy: Correct fields / total
 - Item-level Precision: Correct items
 - Recall: Missed items
- Speed
 - OCR latency: < 1.5s
 - API response: < 2s

Business Metrics

- Successful request: % of valid responses
- End-user accuracy: Manual audit score (if able to obtain data)
- Retry rate: OCR failure rate
- Cost per request: VPS cost/requests

Measures:

- File type validation (only accepts image file type of extension)
- File size limits (reject files that is irrationally large)
- Temporary image deletion after processing
- No long-term storage of client data

17. Maintenance and Cost

These are the projected maintenance and cost:

Subject	Category	Cost (IDR)	Recurring period
VPS Server	Deployment	1.400.000	1 year
API Domain	Deployment	200.000	1 year